

CARTLY ENGINEERING HANDBOOK

A-to-Z Technical Guide

How the React Native shopping app, Express API, authentication, PostgreSQL database, catalogue, pages, state and development workflow work together.

Project: MyApp / Cartly

Edition: 5 August 2026

Environment: React Native Android + Node.js/Express + TypeScript + PostgreSQL + Prisma

Contents

1. Project overview and present status
2. System architecture and request flow
3. Folder structure and responsibilities
4. How the app starts
5. Navigation and every mobile page
6. Contexts and application state
7. Shared API client and offline behavior
8. Registration, OTP, login and token flow
9. Categories, products, search, variants and reviews
10. Express backend architecture
11. PostgreSQL and Prisma data model
12. Complete implemented API reference
13. Product image storage and upload
14. Testing and validation
15. Run commands: USB and wireless
16. Inspecting users and stored data
17. Troubleshooting common errors
18. Security decisions
19. What remains for a production ecommerce app

1. Project overview and present status

Cartly is a React Native retail application inspired by large marketplace apps. The mobile client provides the shopping interface; the Express server owns authentication and catalogue rules; PostgreSQL stores durable data; Prisma provides typed database access. The app is currently Android-tested, but its React Native structure also contains the standard iOS project.

Area	Status	Meaning
Landing, navigation and customer screens	Built	The pages and navigation routes exist and are interactive.
Authentication	Backend connected	Registration, OTP verification, login, refresh, logout and password reset use PostgreSQL.
Catalogue	Backend connected	Categories, products, search, filters, sorting, variants, stock, images, ratings and reviews come from APIs.
Profile and addresses	Backend connected	Profile updates, password changes, account deletion and address management persist in PostgreSQL.

Cart, wishlist and orders	Local state	The user experience exists, but data currently lives in React contexts and disappears after a full app reset.
Checkout, payment, returns and notifications	UI/demo flow	Pages work as a prototype; server APIs and payment processing are still required.
Admin dashboard and production deployment	Not built	Admin APIs/UI, cloud files, hosted database, monitoring and store release remain.

Important: the database schema already models most future ecommerce entities. A table existing in Prisma does not mean its API is implemented. At this edition, authentication, catalogue, account and address modules are real server modules.

2. System architecture and request flow

Samsung Android phone | React Native screens and components | Context providers (auth, catalogue, cart, etc.) | apiClient: JSON + Bearer token + refresh/retry ▼ Express API on laptop, port 4000 | Zod validation → middleware → service | Prisma Client through PostgreSQL adapter ▼ PostgreSQL 18, database: myapp, port 5432

During development, ADB reverse maps the phone’s `localhost:4000` to the laptop’s port 4000 and `localhost:8081` to Metro. This works over USB and Android Wireless Debugging. Therefore the source code can keep using `http://localhost:4000/api/v1` without hard-coding a changing Wi-Fi address.

Typical product request

CategoriesScreen → useCatalogue().listProducts(filters) → catalogueApi.products() → apiGet('/products?...') → Express catalogue route → Zod query validation → catalogueService.products() → Prisma queries PostgreSQL → JSON envelope → API model mapped to mobile CatalogueProduct → FlatList renders ProductCard

3. Folder structure and responsibilities

Path	Purpose
<code>App.tsx</code>	Creates global providers, offline banner, session loading and navigation.
<code>src/navigation</code>	Bottom tabs, root stack and strongly typed route parameters.
<code>src/screens</code>	Complete mobile pages: home, catalogue, product, checkout and account experiences.
<code>components</code>	Navbar, footer and reusable landing-page sections/product cards.
<code>src/context</code>	Global state providers for authentication, catalogue, cart, wishlist, checkout and account data.
<code>src/services</code>	API client, auth/catalogue API wrappers and encrypted token storage.
<code>src/components/feedback</code>	Shared loading button, error/retry screen and offline banner.

<code>backend/src/app.ts</code>	Express app configuration, security middleware, routes, health check and static uploads.
<code>backend/src/modules</code>	Feature modules. Currently <code>auth</code> and <code>catalogue</code> .
<code>backend/src/lib</code>	Prisma, password hashing, JWTs, OTPs, email and shared errors.
<code>backend/prisma/schema.prisma</code>	Database models, relations, enums, indexes and constraints.
<code>backend/prisma/seed.ts</code>	Idempotent development catalogue seed.
<code>backend/uploads/products</code>	Locally uploaded product images for development.

4. How the app starts

1. `index.js` imports `App.tsx` and registers it with React Native.
2. `SafeAreaProvider` handles notches and system bars.
3. `NetworkProvider` subscribes to NetInfo and shows the offline banner.
4. `AuthProvider` reads encrypted tokens from Android Keystore through Keychain.
5. If tokens exist, the backend refresh endpoint rotates them and restores the user.
6. `CatalogueProvider` loads categories and the initial popular product list.
7. Other providers expose local cart, checkout, orders, notifications, search, wishlist and address state.
8. `NavigationContainer` renders the root navigator and initial Home tab.

While the session is being restored, `AsyncStateView` displays a full-screen progress indicator. A network failure presents a retry action without deleting saved credentials.

5. Navigation and every mobile page

Main bottom tabs

Page	What it does	Current data source
Home	Navbar, search, hero carousel, categories, deals, featured products, benefits and footer links.	Catalogue API + UI content
Categories	Server search, categories, price/rating/sale filters, sorting, product cards and empty/retry states.	Catalogue API
Cart	Quantity controls, removal, clear cart, coupon field, totals and checkout navigation.	Local CartContext
Account	Guest login/register entry or signed-in account dashboard and links.	Real auth user + local feature contexts

Shopping stack

Page	Behavior
Search Results	Sends the submitted query to the product API; shows suggestions from the loaded catalogue and stores local search history.

Product Details	Loads product, gallery, variants, stock, ratings, reviews and related products. Add to Cart/Buy Now continue into local cart/checkout.
Wishlist	Shows saved products, supports clear and opens details. Persistence API is pending.
Checkout	Collects delivery address and proceeds through shipping, payment and review.
Shipping Method	Selects the local delivery option.
Payment	Collects a payment selection for the demo flow. No gateway charge is performed.
Order Review	Reviews address, shipping, payment and totals before creating the local order.
Order Success	Displays order confirmation and routes back to shopping/orders.

Authentication and account stack

Page	Behavior
Login	Calls real login API, saves tokens and redirects to Account or Checkout.
Register	Validates fields, creates an unverified database user and requests OTP.
Verify OTP	Verifies email, receives tokens and signs the user in; supports rate-limited resend.
Forgot Password	Requests a reset OTP without revealing whether the email exists.
Reset Password	Consumes reset OTP, hashes the new password and revokes active refresh sessions.
Profile	Updates name/phone in PostgreSQL, changes password with session revocation, and supports password-confirmed account deletion.
Addresses	Authenticated PostgreSQL create/read/update/delete, default selection and automatic replacement default after deletion.
Orders / Details / Tracking	Local order history and status presentation.
Cancellation	Applies local eligibility and cancellation status.
Returns	Creates a local return request and refund-status presentation.
Notifications	Local inbox with mark-all-read and clear.
Help & Support	FAQs and demo support request/actions.
Privacy / Terms	Static legal content selected through one typed route.

6. Contexts and application state

Provider	Responsibilities	Persistence
AuthContext	User, pending registration, login/register/OTP/reset/logout and session restore.	Backend + encrypted tokens
CatalogueContext	Initial categories/products, retry, product cache, detail loading and server list function.	Backend PostgreSQL

NetworkContext	Online status and manual connectivity refresh.	Device network state
CartContext	Items, selections, quantities, coupon and totals.	Memory only
WishlistContext	Saved products, toggle and clear.	Memory only
CheckoutContext	Address form, shipping and payment selections.	Memory only
OrdersContext	Local order creation, lookup, cancellation and returns.	Memory only
AddressBookContext	Loads and mutates the authenticated user's addresses and default selection.	Backend PostgreSQL
NotificationsContext	Local notifications, unread count, mark-read and clear.	Memory only
SearchContext	Recent searches.	Memory only

Contexts prevent prop drilling: a deeply nested screen calls a hook such as `useCart()` instead of receiving cart functions through every parent. For server-backed modules, contexts also centralize caching and shared loading state.

7. Shared API client and offline behavior

`src/services/apiClient.ts` is the single transport layer. Feature wrappers such as `authApi` and `catalogueApi` describe endpoints without repeating fetch/error/token code.

Authenticated request lifecycle

```
Read tokens from Keychain → Attach Authorization: Bearer accessToken → Send request → If success:
parse { success, data } → If 401: use refreshToken once → Store rotated token pair → Retry original
request once → If refresh is invalid: clear tokens and sign out → If refresh failed only due to
network: keep tokens
```

A shared refresh promise prevents five simultaneous expired requests from creating five token rotations. API errors include HTTP status, backend error code and optional details. Network failures become a consistent `NETWORK_ERROR`.

`ConnectivityBanner` uses `@react-native-community/netinfo`. It shows a global red banner when offline and offers Retry. `AsyncButton` provides spinner/disabled behavior; `AsyncStateView` provides full-page loading/error/retry behavior.

8. Registration, OTP, login and token flow

Registration

1. The mobile app posts name, normalized email, ten-digit phone and password.
2. Zod requires valid input and an eight-character minimum password.
3. The server checks for an existing verified email.
4. Password hashing uses Node scrypt with a random salt; plaintext is never stored.
5. A random six-digit OTP is hashed and stored with expiry, purpose, attempts and consumed status.
6. SMTP sends the OTP. Without SMTP in development, it prints in the backend console.

7. Verification consumes the OTP, marks the email verified and issues an access/refresh token pair.

Login and session restoration

Login verifies the script hash and requires verified email. The short-lived JWT access token authorizes APIs; the longer refresh JWT is stored as a hash in PostgreSQL. The raw token pair is saved in Android Keystore via React Native Keychain. On refresh, the old database token is revoked and a new pair is issued. On logout, the refresh token is revoked and device credentials are cleared.

Password reset

The forgot-password response is deliberately generic to prevent email enumeration. A valid reset OTP changes the hash and revokes every active refresh token for that user.

Never inspect or recover plaintext passwords/OTPs: only secure hashes belong in PostgreSQL. Never paste database, SMTP or JWT secrets into chat, source code, screenshots or Git.

9. Categories, products, search, variants and reviews

The seed creates 6 categories, 10 products, stock-aware variants, initial image URLs and 30 review rows. It is idempotent: rerunning it updates matching slugs/SKUs rather than duplicating them.

Filtering and sorting

The server first filters active products by query/category, includes ordered images and variants, calculates minimum/maximum price, total stock, average rating and review count, then applies price/rating/availability/sale filters and sorting. Finally it returns a page with total count, page count and `hasNextPage`.

Supported sort values are `popular`, `rating`, `price_asc`, `price_desc` and `newest`. Search checks name, description and slug case-insensitively. Category filtering supports the category itself and its immediate children.

Mobile model mapping

`catalogueApi.ts` maps database DTOs to the UI's `CatalogueProduct`. Decimal prices are already converted to numbers by the backend. Relative uploaded-image paths are converted to absolute server URLs. A deterministic color and first-letter fallback are generated if no image exists.

Stock

Each SKU variant has independent color, size, price, old price and stock. Product-level stock is the sum of variants, while `inStock` is true when at least one variant has stock. Product Details prevents Add to Cart/Buy Now for an unavailable selected variant. Final stock enforcement must also happen in the future cart/order backend.

Reviews

Review reads are public and paginated. Writes require an access token. The database unique constraint on `(userId, productId)` ensures one review per customer/product; the API uses upsert so resubmission edits it. The mobile details page currently displays server reviews. A review editor UI is still to be added.

10. Express backend architecture

1. `server.ts` creates the HTTP server on `0.0.0.0:4000` and handles graceful shutdown.
2. `app.ts` disables the Express signature and enables Helmet, CORS and the JSON body limit.
3. `/uploads` serves development product files.
4. `/api/health` verifies the API process.
5. `/api/v1` mounts auth and catalogue routers.
6. Routes parse input with Zod, call services and return a standard JSON envelope.
7. Services contain business rules and Prisma operations.
8. The error handler converts Zod, application and upload errors into safe responses.

Response format

```
{
  "success": true,
  "data": { "...": "..." }
}
```

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Request validation failed"
  }
}
```

`authenticate` validates Bearer JWTs and adds `request.auth`. `authorize('ADMIN')` protects image mutation. Authentication and OTP endpoints also use rate limiting.

11. PostgreSQL and Prisma data model

Model	Main responsibility	Important constraints
User	Account, profile, role and email verification.	Unique email and phone; password hash only.
RefreshToken	Server-side refresh sessions.	Unique token hash, expiry and revocation.
VerificationCode	Email verification/password reset OTPs.	Purpose, attempts, expiry and consumed timestamp.
Address	Customer delivery addresses.	User relation and default flag.
Category	Catalogue hierarchy.	Unique slug; self parent/children relation.
Product	Core product copy and active status.	Unique slug and category relation.
ProductImage	Ordered product media.	Product relation and position.
ProductVariant	SKU, color, size, prices and inventory.	Unique SKU.
CartItem	Future persistent customer cart.	One user/variant row.

WishlistItem	Future persistent wishlist.	One user/product row.
Order	Order totals, status, payment and address snapshot.	Unique order number.
OrderItem	Purchased product snapshot.	Stores name/price/variant details at purchase time.
Review	Customer product rating and text.	One review per user/product.
Coupon	Future percentage/fixed discount rules.	Unique code and validity dates.
Notification	Future persistent inbox.	User/date index and read timestamp.
ReturnRequest	Future return/refund workflow.	Status and refund amount.

Deletion cascades are used for user-owned session/OTP/cart/wishlist/review/notification data and product-owned images/variants. Order items store snapshots so historical purchases do not change when current product text or price changes.

12. Complete implemented API reference

Method	Path	Auth	Purpose
GET	/api/health	No	API process health.
POST	/api/v1/auth/register	No	Create/pending account and send verification OTP.
POST	/api/v1/auth/verify-email	No	Verify OTP and issue tokens.
POST	/api/v1/auth/resend-email-otp	No	Rate-limited resend.
POST	/api/v1/auth/login	No	Verify credentials and issue tokens.
POST	/api/v1/auth/refresh	Refresh token body	Rotate session tokens.
POST	/api/v1/auth/logout	Refresh token body	Revoke refresh session.
POST	/api/v1/auth/forgot-password	No	Send reset OTP if account exists.
POST	/api/v1/auth/reset-password	No	Reset password and revoke sessions.
GET	/api/v1/auth/me	Access token	Read current user.
PATCH	/api/v1/account/profile	Access token	Update name and phone.
POST	/api/v1/account/change-password	Access token	Verify current password, update hash and revoke sessions.
DELETE	/api/v1/account	Access token + password	Permanently delete customer data.
GET/POST	/api/v1/addresses	Access token	List or create owned addresses.
PATCH/DELETE	/api/v1/addresses/:addressId	Access token	Update or delete an owned address.
PATCH	/api/v1/addresses/:addressId/default	Access token	Atomically select one default address.

GET	/api/v1/categories	No	Category tree and product counts.
GET	/api/v1/products	No	Search/filter/sort/paginate products.
GET	/api/v1/products/:id-or-slug	No	Product detail.
GET	/api/v1/products/:id/related	No	Related products.
GET	/api/v1/products/:id/reviews	No	Paginated reviews.
POST	/api/v1/products/:id/reviews	Access token	Create/update own review.
POST	/api/v1/products/:id/images	Admin	Upload one image.
DELETE	/api/v1/products/:id/images/:imageId	Admin	Delete image row and local file.

13. Product image storage and upload

Seed products use HTTPS image URLs. New local uploads use multipart field `image`, optional `alt` and optional `position`. Multer writes a randomized file name into `backend/uploads/products`. The server checks declared MIME type, 5 MB limit and binary signature for JPEG/PNG/WebP before creating the database row.

```
POST /api/v1/products/p1/images
Authorization: Bearer ADMIN_ACCESS_TOKEN
Content-Type: multipart/form-data

image      = headphones.png
alt        = Headphones front view
position   = 1
```

The returned path looks like `/uploads/products/uuid.png`. The mobile mapper prefixes the server origin automatically. Local uploads are ignored by Git. For production, replace filesystem storage with S3, Cloudinary or R2 and store the resulting CDN URL.

14. Testing and validation

Command	Coverage	Current result
<code>npm run lint</code>	Mobile ESLint rules.	Passing
<code>npx tsc --noEmit</code>	Mobile TypeScript.	Passing
<code>npm test -- --runInBand</code>	Mobile render and API-client token behavior.	2 suites / 4 tests passing
<code>npm run typecheck</code> in backend	Backend TypeScript.	Passing
<code>npm test</code> in backend	Health and real PostgreSQL catalogue integration tests.	2 suites / 8 tests passing
<code>npm run build</code> in backend	Production TypeScript compilation.	Passing
<code>gradlew app:assembleDebug</code>	Android compile and native autolinking.	Passing

The API-client tests verify Bearer attachment, refresh/retry, token rotation and preserving tokens during network failure. Catalogue tests verify category counts, filtering/sorting/pagination, detail/variants/images, related

products, reviews, invalid queries, authenticated review writing, and admin upload/delete.

15. Run commands: USB and wireless

Prerequisites

- PostgreSQL service `postgresql-x64-18` running.
- Database `myapp` exists and `backend/.env` contains the private connection URL/secrets.
- Phone has Developer Options and debugging enabled.

Terminal 1 — backend

```
cd C:\Users\Preet\OneDrive\Desktop\MyApp\backend
npm run dev
```

Terminal 2 — Metro

```
cd C:\Users\Preet\OneDrive\Desktop\MyApp
npm start
```

Terminal 3 — phone and app

```
adb devices
npm run android:usb
```

`android:usb` forwards ports 8081 and 4000, then builds/launches Android. Despite its name, ADB reverse also works through Wireless Debugging.

Wireless pairing

```
adb pair PHONE_IP:PAIRING_PORT
adb connect PHONE_IP:DEBUG_PORT
adb devices
npm run android:usb
```

The pairing port and debugging port are different and can change after phone restarts. Phone and laptop must share the same Wi-Fi.

Reset Metro cache

```
npm start -- --reset-cache
```

Full Android clean

```
cd android
.\gradlew.bat clean
cd ..
npm start -- --reset-cache
```

Clear all app data (destructive)

```
adb shell pm clear com.myapp
```

This removes login tokens and all local state from the phone.

16. Inspecting users and stored data

```
cd C:\Users\Preet\OneDrive\Desktop\MyApp\backend
npx prisma studio
```

Open <http://localhost:5555>. Use **User** for account rows, **RefreshToken** for login sessions, **VerificationCode** for hashed OTP records, and catalogue tables for categories/products/variants/images/reviews. pgAdmin offers the same data under Databases → myapp → Schemas → public → Tables.

Never manually edit authentication data unless diagnosing development state. Normal changes should go through APIs so validation and security rules run.

17. Troubleshooting common errors

Error	Reason	Fix
Unable to resolve module ./App	App.tsx missing/wrong location/name.	Keep it beside <code>index.js</code> and import <code>./App</code> .
Cannot reach the Cartly server	Backend stopped or ADB 4000 reverse missing.	Start backend; run <code>adb reverse tcp:4000 tcp:4000</code> .
EADDRINUSE 0.0.0.0:4000	Another backend already listens on 4000.	Use the existing process or stop its exact PID before restarting.
Route GET / was not found	Root API route intentionally does not exist.	Use http://localhost:4000/api/health .
Phone absent from adb devices	USB authorization/wireless connection is missing.	Authorize prompt, pair/connect again, then verify device status.
Prisma engine/URL error	Malformed <code>DATABASE_URL</code> or unavailable PostgreSQL.	Check one valid URL line, service status and port 5432.
OTP only appears in console	SMTP is absent.	Add SMTP host/port/user/app-password/from to private <code>.env</code> .
Old JavaScript appears	Metro cache.	Restart using <code>npm start -- --reset-cache</code> .

18. Security decisions

- scrypt salted password hashes; no plaintext passwords.
- OTP hashes with expiry, attempts, purpose and one-time consumption.
- Short access JWTs and rotating, hashed, revocable refresh tokens.
- Device token storage through Keychain/Android Keystore.
- Zod validation on request body/query data.
- Authentication and role authorization middleware.
- Rate limiting for auth and OTP abuse.
- Helmet, controlled CORS, JSON size limit and disabled Express signature.
- Restricted upload MIME, size, filename and binary signature.
- `.env` and uploaded development files excluded from Git.

Production note: card number/CVV must never be stored or processed directly by Cartly. Use a PCI-compliant gateway such as Razorpay or Stripe and verify payments through signed server webhooks.

19. What remains for a production ecommerce app

1. **Persistent cart/wishlist:** guest cart, login merge, quantity/stock checks and server coupons.
2. **Order engine:** server totals, database transaction, inventory reservation/deduction, idempotency and order APIs.
3. **Payments:** gateway checkout, backend verification, webhooks, failure recovery and refunds.
4. **Fulfilment:** status events, tracking, cancellation eligibility, returns, pickup and refund workflow.
5. **Notifications/support:** Firebase push, transactional email, persistent inbox and tickets.
6. **Admin:** dashboard and CRUD for catalogue, inventory, orders, users, coupons, reviews and returns.
7. **Production media:** cloud object storage, resizing, thumbnails, CDN and image deletion jobs.
8. **Operations:** hosted PostgreSQL, HTTPS API, secrets manager, migrations, backups, logs, monitoring and alerts.
9. **Quality:** more screen/component tests, isolated test database, end-to-end checkout tests, accessibility and performance.
10. **Release:** app identity/assets, signing, privacy/account deletion, analytics, crash reporting and Play Store submission.

Recommended immediate next phase: persistent Cart/Wishlist, followed by transactional Orders and Payments.

20. Quick mental model

When learning or extending Cartly, follow one feature vertically. Start at its screen, find the context hook, inspect the API wrapper, follow the Express route into its service, then inspect the Prisma model. Keep UI state in the screen, shared client state in a context, HTTP behavior in services, business rules in backend services and durable facts in PostgreSQL. This separation is what allows the prototype to grow into a maintainable retail system.

Screen → Context → Mobile API wrapper → Shared apiClient → Express route → Validation/Auth → Service → Prisma → PostgreSQL

Cartly A-to-Z Technical Guide · Generated from the project implementation on 5 August 2026. Secrets and private environment values are intentionally excluded.